

RESTful Web API

Lionel Souop

Rappel

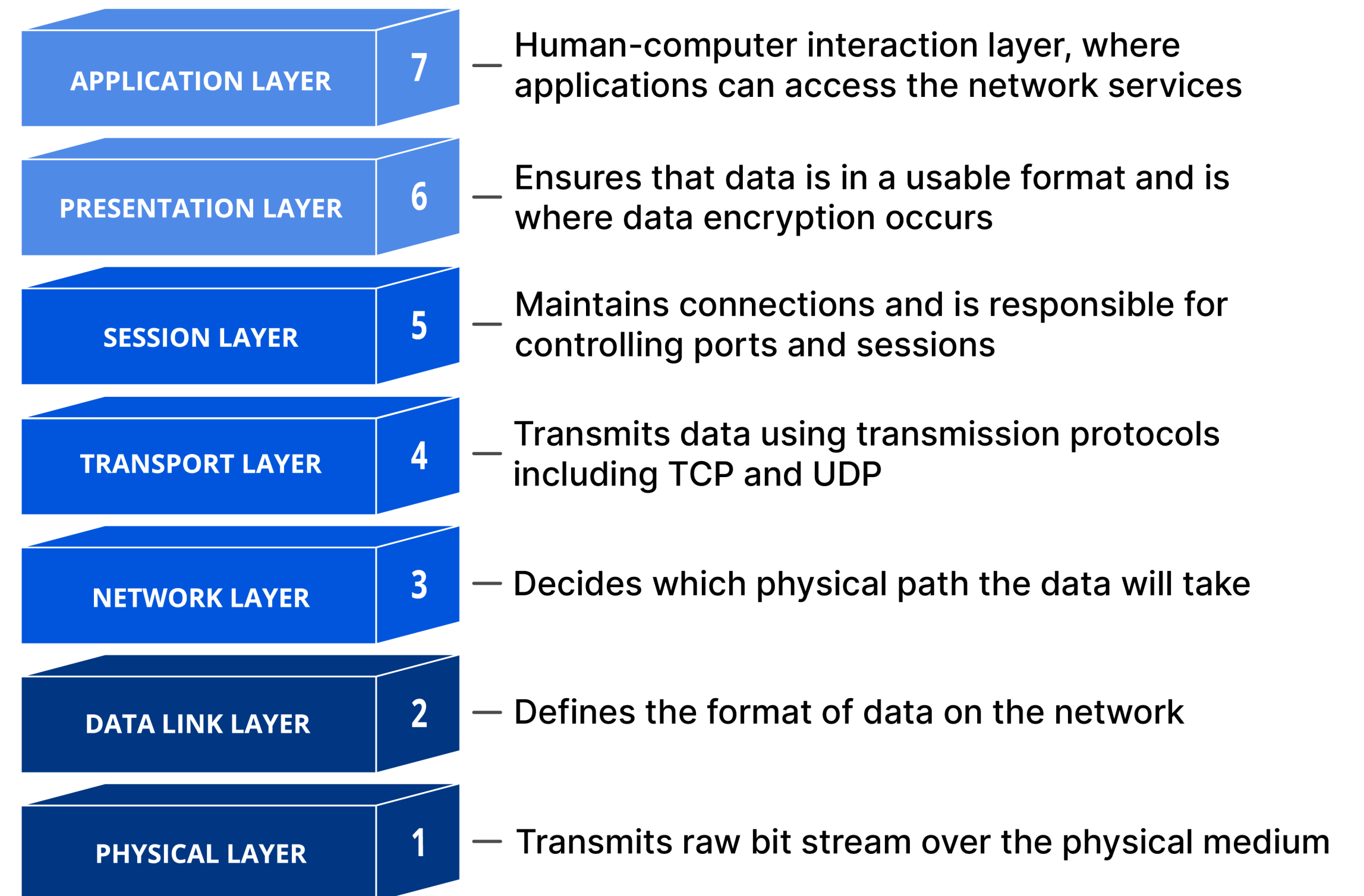
Pile ISO/OSI

- **Un réseau informatique** permet la communication et le partage de ressources entre ordinateurs.
- Exemples : Internet, intranets, réseaux locaux (LAN), réseaux étendus (WAN).

Pourquoi un modèle de référence

- Faciliter la normalisation des communications
- Permettre l'interopérabilité entre systèmes
- Clarifier les rôles des différentes étapes de transmission

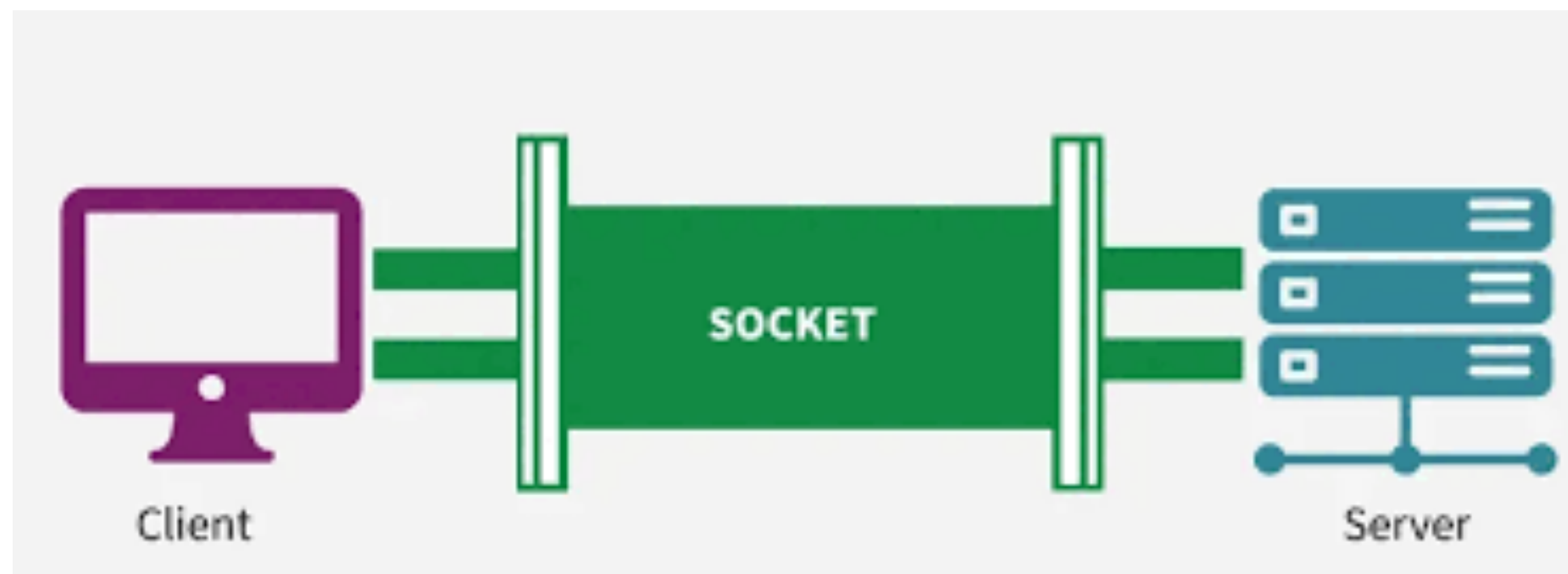
Le modèle **OSI** (Open Systems Interconnection ou interconnexion de systèmes ouverts) est une norme de l'Organisation internationale de standardisation (**ISO**) permettant de définir la communication entre des systèmes informatiques à l'aide de protocoles standards.



Rappel

Socket

- Une socket est une interface logicielle entre une application et le réseau.
- Elle permet la communication entre deux processus (locaux ou distants).



Rappel

Types de Sockets

- Socket TCP (fiable, orientée connexion) - Transport Control Protocol
- Socket UDP (rapide, sans garantie de livraison) - User Datagram Protocol
- Chaque socket est identifiée par une paire (adresse IP, port).

Rappel

TCP - Cycle de vie d'une connexion

Côté serveur :

1. socket()
2. bind()
3. listen()
4. accept()

Côté client :

1. socket()
2. connect()

Rappel

Où se situe le Socket dans OSI

Entre les couches Application (7) et Transport (4) :

- → L'application utilise le Socket pour accéder aux services TCP/UDP.
- → Le système d'exploitation gère la couche transport.

Module Socket en Python

- Le module standard Python permet de créer et d'utiliser des sockets.
- Exemple : `import socket`

Rappel

Serveur/client TCP

serveur

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('localhost', 5000))
s.listen(1)
conn, addr = s.accept()
data = conn.recv(1024)
print(data.decode())
conn.close()
```

Client

```
import socket
c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(('localhost', 5000))
c.send(b'Bonjour serveur !')
c.close()
```


Rappel

UDP Client/serveur

Serveur :

```
socket(AF_INET, SOCK_DGRAM)  
bind(('localhost', 5000))  
recvfrom()
```

Client :

```
socket(AF_INET, SOCK_DGRAM)  
sendto(message, adresse)
```

Rappel

socket.AF_INET et socket.SOCK_STREAM

socket.AF_INET

Signifie : *Address Family Internet (IPv4)*

Rôle : indique que la socket utilisera **le protocole IPv4** (adresses comme 192.168.0.1).

Autres options possibles :

- socket.AF_INET6 → pour **IPv6** (2001:0db8::1)
- socket.AF_UNIX → pour communication **locale** (entre processus sur la même machine, sous Unix)

socket.SOCK_STREAM

Signifie : *Socket de type flux (stream socket)*

Rôle : indique que la socket utilisera le protocole **TCP**

→ c'est-à-dire une **communication fiable**, orientée connexion, avec accusés de réception et ordre garanti.

Autre type fréquent :

- socket.SOCK_DGRAM → pour **UDP** (datagramme)
→ communication **non connectée**, rapide, sans garantie.
-

Rappel

Conclusion

- Le modèle OSI structure la communication en couches.
- Les sockets servent d'interface entre application et réseau.
- TCP garantit la fiabilité, UDP la rapidité.
- Python permet une mise en œuvre simple et pédagogique.

Rappel

Mini TP

Objectif :

Créer un mini-chat client/serveur en Python.

- Échanges bidirectionnels avec sockets TCP
- Interaction par saisie clavier
- Possibilité d'étendre avec threads ou asyncio

API

Définition

API = Application Programming Interface

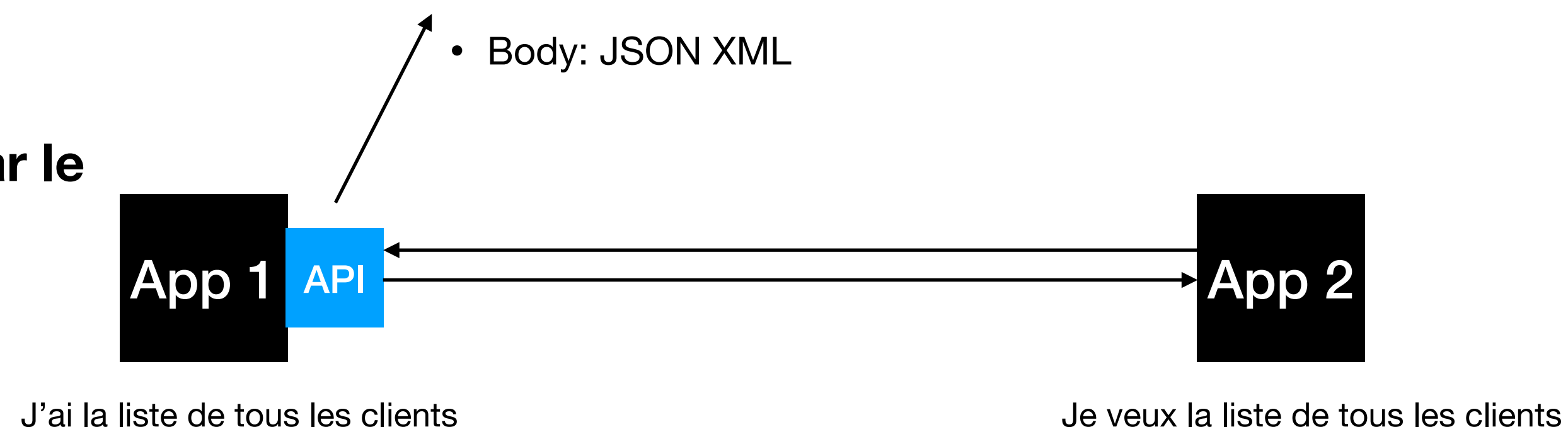
Une API est donc un intermédiaire logiciel qui permet à deux applications de communiquer entre elles

Une API Web peut être considérée comme une passerelle entre les clients et les ressources sur le Web.

Les API sont toutes autour de nous. Chaque fois que nous utilisons une application mobile sur nos téléphones, une API est en jeu

- Définit la méthode de requête: POST, GET, PUT, DELETE, HEAD, CONNECT etc
- Expose une URL (Uniform Resource Locator) communément appelé adresse Web
- Mécanisme d'authentification: user-password/private-public-key/certificate
- Body: JSON XML

Applications qui communiquent par le biais d'une API Web



API

Définition

La communication entre 2 applications est forcément initiée par une des applications

La communication via API suit le modèle client-server (le client dans certains cas peut aussi être serveur et vice versa)

Les API sont principalement utilisées pour les services Web

La communication via socket entre process peut aussi être assimilée aux API

Objectifs

- SOAP
- REST
- Construire une API RESTFul

SOAP

Définition

- **S**imple **O**bject **A**ccess **P**rotocol
 - SOAP est un protocole de communication pensé pour Internet
 - SOAP peut s'étendre à l'HTTP pour des messages XML.
 - SOAP Fournit le transport des données pour les services Web.
 - SOAP permet de transférer un document complet ou appeler une RPC
 - SOAP peut être utilisé pour la diffusion de messages.
 - SOAP est indépendant de la plateforme et du langage de programmation.
 - SOAP s'appuie sur l'XML pour la définition des contenus à envoyer sur le réseau.
 - SOAP permet une connection facile entre client et serveur

SOAP

Message

- Un message SOAP est un XML constitué des éléments ci-après:
 - Enveloppe - élément obligatoire qui définit le début et la fin du message
 - Header - contient les attributs optionnel du message
 - Body - élément obligatoire qui contient le message à envoyer
 - Fault - élément optionnel qui contient des informations les erreurs survenues lors de la manipulation du message

SOAP Message

```
<?xml version = "1.0"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV = "http://www.w3.org/2001/12/soap-envelope"
  SOAP-ENV:encodingStyle = "http://www.w3.org/2001/12/soap-encoding">

  <SOAP-ENV:Header>
    ...
    ...
  </SOAP-ENV:Header>
  <SOAP-ENV:Body>
    ...
    ...
    <SOAP-ENV:Fault>
      ...
      ...
    </SOAP-ENV:Fault>
    ...
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

SOAP

Envelope

- L'élément enveloppe indique le début et la fin d'un message pour que le destinataire sache quand le message est entièrement reçu.
- Chaque message SOAP a un element Enveloppe racine
- Chaque Enveloppe contient un seul élément body
- Si l'enveloppe contient le Header, il peut en contenir plusieurs et il sont les premiers éléments avant le Body

SOAP

Envelope

```
<?xml version = "1.0"?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV = "http://www.w3.org/2001/12/soap-envelope"
  SOAP-ENV:encodingStyle = " http://www.w3.org/2001/12/soap-encoding">
  ...
  Message information goes here
  ...
</SOAP-ENV:Envelope>
```

SOAP

Envelope - SOAP with HTTP POST

```
POST /OrderEntry HTTP/1.1
Host: www.tutorialspoint.com
Content-Type: application/soap; charset="utf-8"
Content-Length: nnnn

<?xml version = "1.0"?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV = "http://www.w3.org/2001/12/soap-envelope"
  SOAP-ENV:encodingStyle = " http://www.w3.org/2001/12/soap-encoding">
  ...
  Message information goes here
  ...
</SOAP-ENV:Envelope>
```

Illustration de l'utilisation des messages SOAP avec du HTTP POST.

Il montre le namespace du schema de l'enveloppe: xmlns = xml namespace

Il montre aussi les règles d'encodage

SOAP

Header

- Élément optionnel qui offre plus de flexibilité au framework pour fournir plus d'exigence, plus de paramètres
- Le header peut être utilisé pour spécifier la signature électronique, des métadonnées, la technique de cryptage à utiliser pour accéder à un système distant.
- Il peut avoir plusieurs Header
- Le header à pour but d'apporter une nouvelle fonctionnalité
- L'élément header est défini dans un namespace
- Le header est le premier enfant de l'élément Enveloppe

SOAP

Header

```
<?xml version = "1.0"?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV = " http://www.w3.org/2001/12/soap-envelope"
  SOAP-ENV:encodingStyle = " http://www.w3.org/2001/12/soap-encoding">

  <SOAP-ENV:Header>
    <t:Transaction
      xmlns:t = "http://www.tutorialspoint.com/transaction/"
      SOAP-ENV:mustUnderstand = "true">5
    </t:Transaction>
  </SOAP-ENV:Header>
  ...
  ...
</SOAP-ENV:Envelope>
```

- mustUnderstand est true/false si la présence d'un header est optionnel ou obligatoire

SOAP

Body

- Élément obligatoire qui contient le message XML à envoyer dans le message SOAP
- Le Body se définit dans l'enveloppe
- Le body contient des information à l'intention du destinataire du message

```
<?xml version = "1.0"?>
<SOAP-ENV:Envelope>
  .....
  <SOAP-ENV:Body>
    <m:GetQuotation xmlns:m = "http://www.tp.com/Quotation">
      <m:Item>Computers</m:Item>
    </m:GetQuotation>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

```
<?xml version = "1.0"?>
<SOAP-ENV:Envelope>
  .....
  <SOAP-ENV:Body>
    <m:GetQuotationResponse xmlns:m = "http://www.tp.com/Quotation">
      <m:Quotation>This is Qutation</m:Quotation>
    </m:GetQuotationResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```


SOAP

Body

- SOAP with a double data type

```
<?xml version = '1.0' encoding = 'UTF-8'?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV = "http://www.w3.org/2001/12/soap-envelope"
  xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd = "http://www.w3.org/2001/XMLSchema">

  <SOAP-ENV:Body>
    <ns1:getPriceResponse
      xmlns:ns1 = "urn:examples:priceservice"
      SOAP-ENV:encodingStyle = "http://www.w3.org/2001/12/soap-encoding">
      <return xsi:type = "xsd:double">54.99</return>
    </ns1:getPriceResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

SOAP

Transport

Bien que le HTTP soit le protocole de transport préféré pour les message SOAP, il n'est lié à aucun protocole de transport en particulier, et peut donc être utilisé via SMTP, FTP etc ...

Pour accéder au service de traduction AltaVista BabelFish, hear XMethods, il faut spécifier une SOAPAction dans le header.

```
POST /perl/soaplite.cgi HTTP/1.0
Host: services.xmethods.com
Content-Type: text/xml; charset = utf-8
Content-Length: 538
SOAPAction: "urn:xmethodsBabelFish#BabelFish"

<?xml version = '1.0' encoding = 'UTF-8'?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV = "http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi = "http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd = "http://www.w3.org/1999/XMLSchema">

  <SOAP-ENV:Body>
    <ns1:BabelFish
      xmlns:ns1 = "urn:xmethodsBabelFish"
      SOAP-ENV:encodingStyle = "http://schemas.xmlsoap.org/soap/encoding/">
      <translationmode xsi:type = "xsd:string">en_fr</translationmode>
      <sourcedata xsi:type = "xsd:string">Hello, world!</sourcedata>
    </ns1:BabelFish>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

SOAP

Transport

```
HTTP/1.1 200 OK
Date: Sat, 09 Jun 2001 15:01:55 GMT
Server: Apache/1.3.14 (Unix) tomcat/1.0 PHP/4.0.1pl2
SOAPServer: SOAP::Lite/0.50
Cache-Control: s-maxage = 60, proxy-revalidate
Content-Length: 539
Content-Type: text/xml

<?xml version = "1.0" encoding = "UTF-8"?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENC = "http://schemas.xmlsoap.org/soap/encoding/"
  SOAP-ENV:encodingStyle = "http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:xsi = "http://www.w3.org/1999/XMLSchema-instance"
  xmlns:SOAP-ENV = "http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd = "http://www.w3.org/1999/XMLSchema">

  <SOAP-ENV:Body>
    <namesp1:BabelFishResponse xmlns:namesp1 = "urn:xmethodsBabelFish">
      <return xsi:type = "xsd:string">Bonjour, monde!</return>
    </namesp1:BabelFishResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```


REST

Définition

- **Representational State Transfer**
- Protocol de communication pour les services Web
- S'appuie fortement sur les méthodes HTTP POST/GET/PUT/DELETE
- Utilise principalement le JSON pour le transfer de données
 - Le XML aussi peut être utilisé
- Stateless: chaque demande d'un client au serveur est indépendante et autonome, ce qui signifie que le serveur ne conserve pas les informations de session entre les demandes
- Scalabilité - facilement scalable horizontalement
- Supporte les mécanismes de caching

API REST

Lorsqu'on parle de REST, on utilise souvent le modèle de maturité de Richardson qui se compose de 4 niveaux, pour décrire les règles à respecter lors de la mise en place d'une API REST:

- Niveau 0. Swamp of POX
- Niveau 1: Resources
- Niveau 2: Utilisation des verbes et entêtes HTTP
- Niveau 3: Hypermedia controls

REST API

Niveau 0

Expose une seule URI,

utilisation du POST pour toutes les actions (méthodes à appeler dans le body)

SOAP opère à ce niveau

REST API

Niveau 1

Utilisation des URI pour désigner les ressources individuelles comme des services:

Exemple:

/users, /users/123

/customers, /customers/456, /customers/456/orders

REST API

Niveau 2

Utilisation des verbes et entêtes HTTP pour les interactions avec les ressources

- Get, post, put, delete, patch, options
- Accept, content-types, authorizations, if-match etc ...

REST API

Niveau 3

Hypermédia comme moteur d'application (HATEOAS: Hypertext as the engine of application state).

La ressource indique comment accéder aux autres ressources et quelles sont les actions possible sur la ressources - comme une doc.

Liée les ressources entre elles à travers des liens

Pas besoin pour le client de se fier à une doc externe,

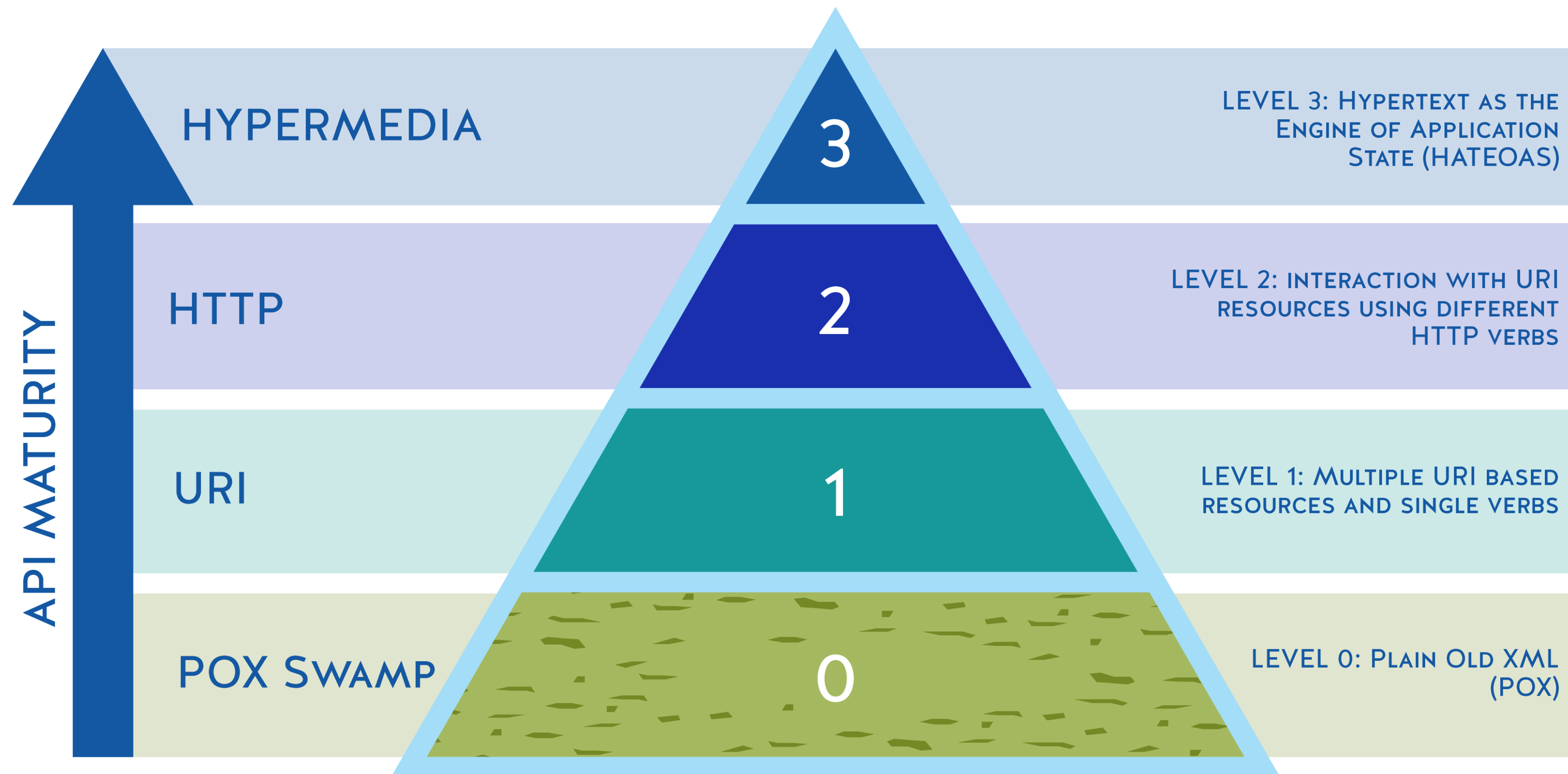
Ce qui permet au serveur de changer ses URI sans briser le client

```
GET /status HTTP/1.1
Accept: application/json
-----
HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_embedded": {
    "status": []
  },
  "_links": {
    "self": {
      "href": "http://localhost/posts"
    }
  },
  "page_count": 0,
  "page_size": 10,
  "total_items": 0
}
```

API REST

RICHARDSON MATURITY MODEL



API REST

Négociation de contenu

Il est conseillé de gérer au minimum 2 formats: JSON et XML.

L'API pourra distribuer les ressources au format JSON et fournir aussi du XML pour les utilisateurs de ce format

API REST

Sécurité

- SSL
- OAuth2
- JWT

REST & RESTFul

- REST est un style d'architecture
- RESTFul est une application “full of REST”

RESTFUL VS RESTLESS

RESTLESS WEB SERVICE

RESTFUL WEB SERVICE	RESTLESS WEB SERVICE
An application that conforms to the REST architectural style that provides interoperability between computer systems on the internet	An application that is not based on the principles of REST
Use REST	Use SOAP
Support various data formats such as HTML, JSON, text etc.	Support XML format
Use URL to expose business logic	Use the service interface to expose business logic
Easier and flexible	Not as easy or flexible
Inherits security measures from the underlying transport protocols; therefore, it is less secure	Defines its own security layer and is more secure
Consume less bandwidth and resource	Consume more bandwidth and resources

Django REST framework

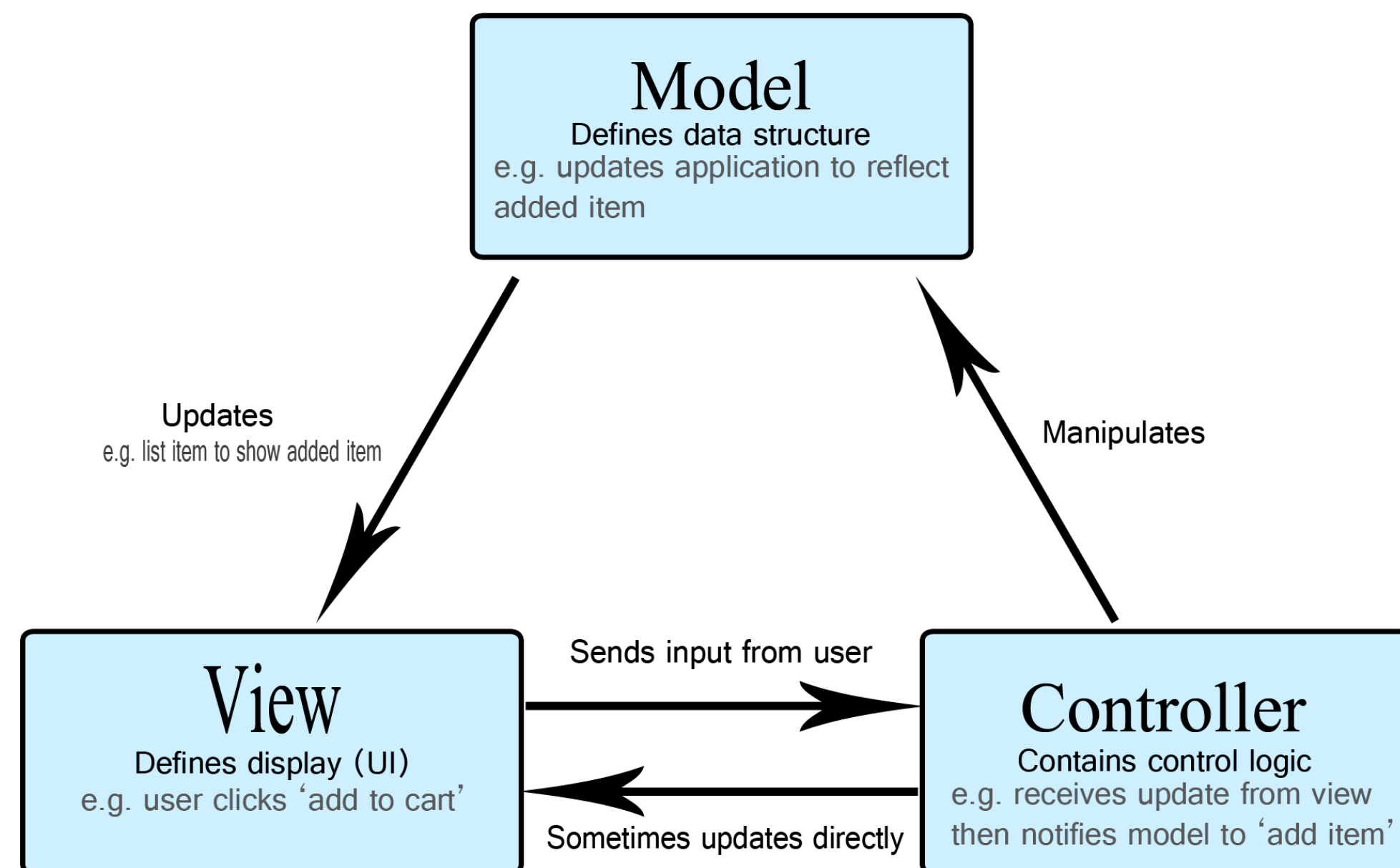
REST frameworks

- Django
- Flask
- Express.js
- Spring Boot
- Etc ...

Model View Controller

MVC

Le MVC est un modèle d'architecture logicielle qui fournit une façon d'organiser une application. Elle consiste à distinguer trois entités distinctes qui sont, le **modèle**, la **vue** et le **contrôleur** ayant chacun un rôle précis.



- Histoire et évolution des API
- Leur importance pour les data lakes
- Les protocoles api
- Communication via socket est à l'origine - des TP pour ça??